

AULA 09 • PROGRAMAÇÃO ORIENTADA A OBJETOS

Collections: List

Trocar o array de tamanho fixo por listas que crescem sozinhas —
e ordenar tudo com `Collections.sort()`.

Curso de Java • Prof. Rob Silva

O caminho de hoje.

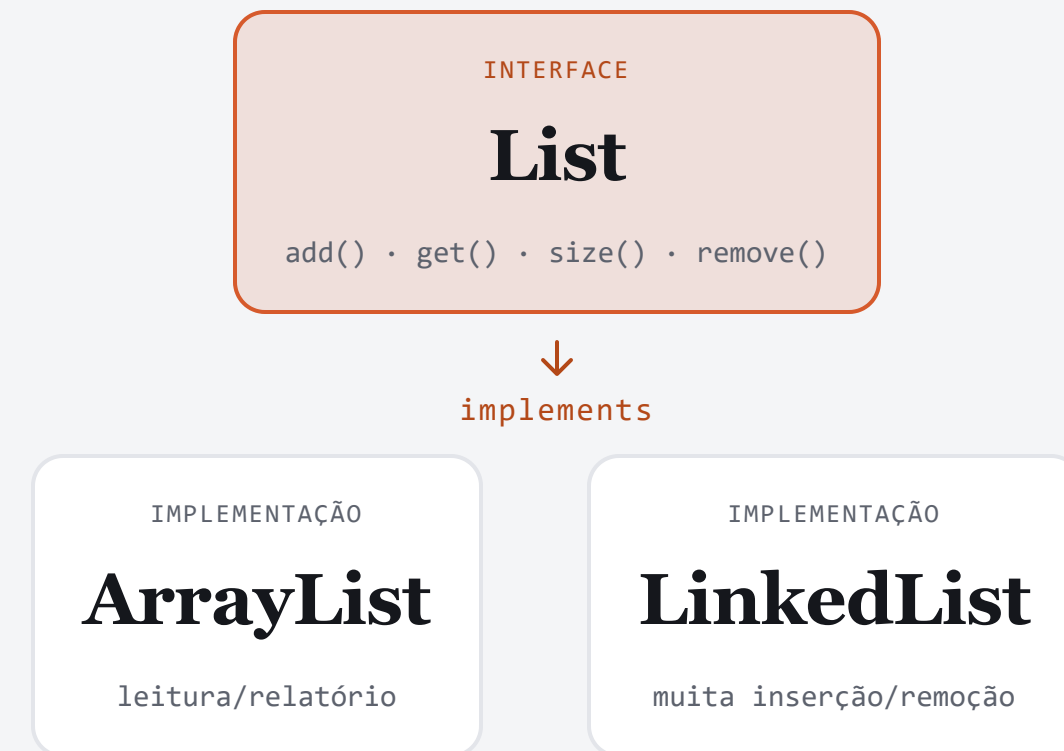
- 01 O problema: array tem tamanho fixo
- 02 Collections Framework e a interface `List`
- 03 `ArrayList` : add, get, size, remove
- 04 Percorrer a lista — três formas
- 05 Ordenar com `Comparable` e `Collections.sort()`
- 06 Tarefa da aula — ticket JAVA-109

O array resolve — até precisar crescer.

- Na `Aula 7` vimos herança e polimorfismo; na `Aula 8`, interfaces e listas de tipos diferentes.
- O estoque ainda usa `Produto[]`: você declara o tamanho uma vez e ele **não cresce**.
- Faltou espaço? Recriar o array maior e copiar tudo. Há um jeito melhor.

Estruturas prontas para guardar objetos.

- O **Collections Framework** traz coleções prontas do Java.
- `List` é uma coleção **ordenada** que aceita repetidos e cresce sozinha.
- `ArrayList` é a implementação mais usada de `List`.



Uma lista que começa vazia e cresce.

```
import java.util.ArrayList;
import java.util.List;

List<String> produtos = new ArrayList<>();
produtos.add("Notebook");
produtos.add("Mouse");
produtos.add("Teclado");

System.out.println(produtos.size()); // 3
System.out.println(produtos.get(0)); // Notebook
produtos.remove("Mouse");
```

Sem tamanho fixo: `add` insere, `size()` conta, `get(i)` lê por índice e `remove` tira o item.

A variável é do tipo **List**; o objeto é um **ArrayList**.

Qual List usar?

ArrayList

Acesso por índice rápido (`get`). Ótimo para leitura e relatórios. É a escolha padrão.

LinkedList

Inserção e remoção no início ou no meio são mais eficientes. Útil em filas.

No estoque

A gente lê e lista muito mais do que insere no meio. `ArrayList` resolve.

Três formas de visitar cada item.

```
// 1) índice
for (int i = 0; i < produtos.size(); i++) {
    System.out.println(produtos.get(i));
}

// 2) for-each
for (String p : produtos) {
    System.out.println(p);
}

// 3) Iterator
Iterator<String> it = produtos.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

O mesmo resultado por três caminhos. O `for-each` é o mais limpo no dia a dia.

Três formas, quando usar cada.

índice

Use quando precisar da **posição** do item ou andar de trás para frente.

for-each

O padrão para apenas **ler** cada elemento. Curto e difícil de errar.

Iterator

Quando precisar **remover** itens com segurança enquanto percorre.

ORDENAÇÃO • INTRODUÇÃO

Ordenar sem reinventar a roda.

Você não precisa escrever bubble sort. `Collections.sort()`
ordena a lista inteira — basta dizer ao Java **qual** é a ordem
natural dos seus objetos.

A ordem natural do objeto.

- `Collections.sort()`
precisa saber: o que vem antes, o que vem depois?
- Implementar `Comparable`
define a **ordem natural** da sua classe.
- O método `compareTo()`
compara dois objetos:
negativo, zero ou positivo.

Ensinando o Produto a se comparar.

```
public class Produto implements Comparable<Produto> {  
    private String nome;  
    private double preco;  
    private int quantidade;  
    private String categoria;  
  
    // construtor e getters...  
  
    @Override  
    public int compareTo(Produto outro) {  
        return this.nome.compareTo(outro.nome); // ordem alfabética  
    }  
}
```

É o `Produto` da MegaStore, agora `implements Comparable<Produto>`. O `compareTo` delega para o `compareTo` da `String` — ordem alfabética por nome.

Uma linha ordena tudo.

```
List<Produto> estoque = new ArrayList<>();
estoque.add(new Produto("Teclado", 120.0, 30, "Periféricos"));
estoque.add(new Produto("Notebook", 3500.0, 15, "Eletrônicos"));

Collections.sort(estoque); // usa compareTo (nome)
```

Como `Produto` implementa `Comparable`, o `sort` sabe ordenar sozinho. Depois desta linha, **Notebook** vem antes de **Teclado**.

Do array fixo para a lista que cresce.



- O menu `Scanner` (1/2/3/0) continua igual.
- O cadastro vira `estoque.add(...)`; a exclusão vira `estoque.remove(...)`.
- A opção 3 ordena com `Collections.sort()` e percorre com `for-each`.

Cadastrar, excluir e listar em ordem.

```
List<Produto> estoque = new ArrayList<>();

// CREATE
estoque.add(new Produto("Notebook", 3500.0, 15, "Eletrônicos"));

// DELETE por índice
estoque.remove(0);

// RELATÓRIO ordenado
Collections.sort(estoque);
for (Produto p : estoque) {
    System.out.println(p.descricao());
}
```

O CRUD do `JAVA-104` agora roda sobre uma `List`: `add` cadastra, `remove` exclui e o relatório sai **ordenado** com um `for-each`.

Três ideias para levar para casa.

List / ArrayList

Coleção que cresce sozinha. `add`, `get`, `size` e `remove` no lugar do array fixo.

iteração

Índice, `for-each` ou `Iterator`. Para só ler, prefira o `for-each`.

ordenação

`Comparable` define a ordem natural; `Collections.sort()` ordena a lista inteira.

JAVA-109

MÉDIO

TO DO

BACKEND

RESPONSÁVEL: VOCÊ

Estoque em lista que cresce e relatório ordenado

HISTÓRIA DO USUÁRIO

Como gerente da MegaStore, quero que o sistema guarde os produtos em uma **lista que cresce sozinha** e exiba o relatório em **ordem alfabética**, para não depender mais de um array de tamanho fixo.

CONTEXTO

Evolui o JAVA-104 : hoje o estoque usa `Produto[]` com tamanho fixo. Troque por `ArrayList<Produto>` e ordene o relatório.

JAVA-109

MÉDIO

PRONTO QUANDO TODOS OS ITENS PASSAREM

CRITÉRIOS DE ACEITE

- ☒ Substituir o `Produto[]` por `ArrayList<Produto>` no CRUD do estoque.
- ☒ `add` e `remove` funcionando nas opções de cadastro e exclusão.
- ☒ `Produto` implements `Comparable<Produto>`, ordenando por `nome`.
- ☒ A opção 3 (Relatório) usa `Collections.sort()` e imprime em ordem alfabética.
- ☒ Percorrer a lista com `for-each`. O menu e o `Scanner` do `JAVA-104` continuam iguais.

DESAFIO EXTRA

Ordene o relatório por `preco` em vez de `nome` usando um `Comparator` (dica: `Comparator.comparingDouble`) — opcional.

#megastore #java-104 #collections #entrega: próxima aula

Resolva usando apenas `List/ArrayList`, iteração e `Collections.sort()` com `Comparable` – nada além do visto na Aula 9.

